

Casos de Teste POC — Validação da Plataforma de Dados



Huawei Technologies Co., Ltd.

v2.4.0 — 27 de setembro de 2026 17:34

Sumário

1	Introdução	2
1.1	Escopo dos Testes.....	2
1.2	Pré-condições e Preparações	2
1.3	Método de Aceitação	3
2	Casos de Teste	4
2.1	Arquitetura da Plataforma	4
2.2	Engenharia de Dados	7
2.3	Segurança e Controle de Acesso	10
2.4	Configuração de Ambiente e Ferramentas	13
3	Conclusão	16
3.1	Diagramas de Arquitetura de Teste.....	16
3.2	Paleta de Cores da Marca	18
4	Histórico de versões	19

Informação

Este documento demonstra o template testbook (`testbook.cls`) para documentos de casos de teste POC/aceitação do Huawei Cloud. Ele exercita todos os comandos e ambientes disponíveis: capa, sumário, ambiente de caso de teste com todos os comandos de campo, barras de cabeçalho mini, ambiente testprocedure com ações passo a passo, bloco de objetivos, blocos de código, imagens, caixas de destaque (`warning`, `tip`, `infobox`), tabelas, badge, caminhos de menu, weblinks, notas, referências param, tabela de resumo dos testes, result badges e histórico de versões.

Novo

Objetivo Geral: Validar a implantação da Plataforma de Dados do Huawei Cloud para o ambiente POC, garantindo que todos os serviços principais (DataArts Studio, MRS, OBS, VPC) estejam provisionados corretamente, acessíveis e funcionando conforme especificado na arquitetura da solução.

1.1 Escopo dos Testes

Domínio	O Que Deve Ser Observado
Arquitetura da Plataforma	Serviços principais estão implantados, saudáveis e acessíveis via API do Console.
Engenharia de Dados	Pipelines do DataArts Studio conseguem ingerir, transformar e carregar dados no armazenamento de destino.
Armazenamento de Dados	Buckets OBS e HDFS do MRS aceitam escritas/leitura com permissões corretas.
Segurança	Políticas IAM, isolamento VPC e regras de grupo de segurança impõem o controle de acesso pretendido.

Tabela 1: Escopo dos testes por domínio.

1.2 Pré-condições e Preparações

- Todos os recursos de infraestrutura devem ser provisionados via Terraform e a saída do `terraform apply` deve apresentar zero erros.
- O cluster MRS deve estar no estado **Executando** antes da execução de qualquer caso de teste de engenharia de dados.
- Arquivos de dados de teste (CSV/JSON) devem ser carregados no bucket OBS designado antes da execução dos pipelines.
- Cada testador deve ter uma conta IAM pessoal com a função **Dayu_User** atribuída no DataArts Studio.
- Regras de firewall de rede devem permitir TCP 22 (SSH) e TCP 443 (HTTPS) de entrada a partir da sub-rede de execução de testes.

Nota: Certifique-se de que o ambiente de execução de testes esteja configurado com o fuso horário *America/Sao_Paulo (GMT-3)* para que os registros de data/hora nos logs estejam alinhados com o Console do Huawei Cloud.

```
# Verificar se as ferramentas CLI estão instaladas
terraform version
obsutil version
kubectl version --client
```

Dica

Utilize `terraform plan` antes de `terraform apply` para visualizar as alterações de infraestrutura e evitar modificações indesejadas de recursos.

1.3 Método de Aceitação

Resultado	Amostra	Explicação
Atende	Atende	O caso de teste passa completamente — o comportamento observado corresponde ao resultado esperado sem desvios.
Atende com ressalvas	Atende com ressalvas	O caso de teste foi executado com sucesso, mas com restrições, dependências ou configuração adicional (documentadas em Observações).
Não atende	Não atende	O caso de teste falha — o comportamento observado não corresponde ao resultado esperado. Um relatório de defeito deve ser registrado.
Não testado	Não testado	O caso de teste não foi executado — a execução permanece pendente e deve ser agendada.

Tabela 2: Critérios de aceitação.

2.1 Arquitetura da Plataforma

Caso de Teste 1: Verificar implantação e saúde do cluster MRS

Objetivo

Confirmar que o cluster MapReduce Service (MRS) está implantado com os componentes necessários (HDFS, YARN, Spark, Hive) e que todos os nós apresentam status saudável.

Escopo do Teste

Implantação do cluster MRS; Monitoramento de saúde; Verificação de status dos nós

Pré-requisitos

1. Aplicação da infraestrutura Terraform concluída com sucesso.
2. ID do cluster MRS disponível na saída do Terraform.
3. Usuário IAM possui função **MRS_Viewer** ou superior.

Procedimento

1. Acessar o Console do Huawei Cloud
2. Navegar em **Console** → **MapReduce Service** → **Clusters**
3. Localizar cluster `mrs-poc-cluster`
4. Verificar status do cluster
5. Revisar aba Componente
6. Revisar aba Nó



Resultado Esperado

1. Status do cluster é **Executando**.

2. Todos os quatro componentes (HDFS, YARN, Spark, Hive) estão listados com status **Normal**.
3. Todos os nós apresentam status **Executando** sem alarmes.

Resultado do Teste

Atende

Observações

Se algum componente apresentar status **Anormal**, verificar a página de alarmes do MRS antes de registrar o resultado.

Caso de Teste 2: Verificar criação e acessibilidade do bucket OBS

Objetivo

Confirmar que o bucket do Object Storage Service (OBS) utilizado para o data lake da POC está criado, possui a classe de armazenamento correta e é acessível com a política IAM designada.

Escopo do Teste

Criação de bucket OBS; Acessibilidade de dados; Configuração de armazenamento

Pré-requisitos

1. Nome do bucket OBS definido na configuração Terraform.
2. Política IAM concedendo acesso de leitura/escrita ao bucket está anexada ao usuário de teste.

Procedimento

1. Acessar o Console
2. Navegar em **Console** → **Object Storage Service**
3. Localizar o bucket (ex.: poc-data-lake-raw)
4. Verificar se a classe de armazenamento é **Standard**
5. Carregar um arquivo de teste (test-upload.txt) no bucket
6. Baixar o arquivo e comparar seu conteúdo com o original
7. Excluir o arquivo de teste

Resultado Esperado

1. O bucket existe com classe de armazenamento **Standard**.

2. O carregamento do arquivo conclui sem erro.
3. O conteúdo do arquivo baixado corresponde ao original.
4. A exclusão do arquivo é realizada com sucesso.

Resultado do Teste

Atende

Observações

A consistência eventual do OBS pode causar um breve atraso antes que um objeto recém-carregado apareça nas listagens.

Caso de Teste 3: Verificar configuração de VPC e grupo de segurança

Objetivo

Confirmar que a VPC, a sub-rede e os grupos de segurança estão configurados conforme o diagrama de arquitetura de rede, com os blocos CIDR e regras de entrada/saída corretos.

Escopo do Teste

Configuração de VPC; Configuração de subnet; Regras de security group

Pré-requisitos

1. IDs da VPC e da sub-rede disponíveis na saída do Terraform.
2. Nomes dos grupos de segurança documentados na arquitetura da solução.

Procedimento

1. Acessar o Console
2. Navegar em **Console** → **Virtual Private Cloud** → **VPCs**
3. Localizar a VPC (ex.: vpc-poc) e verificar se o bloco CIDR corresponde à arquitetura (ex.: 10.0.0.0/16)
4. Clicar na sub-rede e verificar seu CIDR (ex.: 10.0.1.0/24)
5. Navegar até **Security Groups** e verificar regras de entrada: TCP 22 da sub-rede bastion, TCP 443 do proxy corporativo
6. Verificar se as regras de saída permitem todo o tráfego (padrão)

Resultado Esperado

1. CIDR da VPC é 10.0.0.0/16.

2. CIDR da sub-rede é 10.0.1.0/24.
3. Regras de entrada correspondem à arquitetura: TCP 22 e TCP 443 das origens especificadas.
4. Saída permite todo o tráfego.

Resultado do Teste

Atende

Observações

Se o grupo de segurança possuir regras adicionais além da especificação da arquitetura, documentá-las no campo de Observações.

2.2 Engenharia de Dados

Caso de Teste 4: Verificar criação do workspace do DataArts Studio

Objetivo

Confirmar que a instância do DataArts Studio (Dayu) está provisionada, o workspace está acessível e os usuários IAM designados conseguem acessar com a função **Dayu_User**.

Escopo do Teste

Provisionamento do DataArts Studio; Acesso ao workspace; Atribuição de função IAM

Pré-requisitos

1. Instância do DataArts Studio provisionada e com status **Executando**.
2. Usuário IAM de teste possui a função **Dayu_User** atribuída.

Procedimento

1. Acessar o Console com o usuário IAM de teste
2. Navegar em **Console** → **DataArts Studio** → **Workspaces**
3. Localizar o workspace (ex.: poc-workspace)
4. Clicar em **Acessar Workspace** para abrir o console do DataArts Studio
5. Verificar se o painel de navegação à esquerda carrega com os módulos esperados: **Integração de Dados**, **Desenvolvimento de Dados**, **Arquitetura de Dados**

Resultado Esperado

1. O workspace está listado e acessível.
2. O console do DataArts Studio abre sem erros.
3. Todos os três módulos (Integração de Dados, Desenvolvimento de Dados, Arquitetura de Dados) estão visíveis na navegação.

Resultado do Teste

Atende

Observações

Se o workspace falhar ao carregar, verificar o status da instância Dayu e a atribuição da função IAM.

Caso de Teste 5: Verificar execução do pipeline de ingestão de dados

Objetivo

Confirmar que um pipeline em lote do DataArts Studio consegue ingerir dados CSV do OBS, aplicar uma transformação básica e gravar o resultado no caminho OBS de destino.

Escopo do Teste

Pipeline de ingestão de dados; Transformação OBS-to-OBS; Execução de pipeline de dados

Pré-requisitos

1. Caso de Teste 4 concluído (workspace do DataArts Studio acessível).
2. Arquivo CSV de origem (customers.csv) existe no bucket OBS raw.
3. Caminho OBS de destino (poc-data-lake-curated/customers/) está configurado.
4. Pipeline pl-ingest-customers está publicado no DataArts Studio.

Procedimento

1. Navegar até Desenvolvimento de Dados no DataArts Studio
2. Localizar pipeline pl-ingest-customers
3. Clicar em Executar para iniciar o pipeline
4. Aguardar status do pipeline (timeout: 10 min)
5. Verificar arquivo de saída no OBS

6. Baixar e verificar contagem de linhas

```
# Verificar o arquivo de saída no OBS via CLI
obsutil ls obs://poc-datalake-curated/customers/ -limit=10
obsutil stat obs://poc-datalake-curated/customers/output.csv
```

Resultado Esperado

1. A execução do pipeline conclui com status **Sucesso**.
2. O arquivo de saída é criado no caminho OBS de destino.
3. A contagem de linhas do arquivo de saída corresponde ao arquivo de origem.

Resultado do Teste

Atende

Observações

O tempo de execução do pipeline varia conforme o volume de dados. Para a POC, o arquivo de origem contém aproximadamente 10.000 linhas.

Caso de Teste 6: Verificar consulta Spark SQL no cluster MRS

Objetivo

Confirmar que uma consulta Spark SQL pode ser executada no cluster MRS para ler dados de uma tabela Hive e retornar resultados corretos, validando a integração entre o MRS e o data lake.

Escopo do Teste

Execução Spark SQL; Integração com tabela Hive; Validação do data lake MRS

Pré-requisitos

1. Caso de Teste 1 concluído (cluster MRS saudável).
2. Tabela Hive `poc_db.customers` existe e contém dados carregados pelo pipeline de ingestão.
3. Acesso SSH ao nó master do MRS configurado.

Procedimento

1. Acessar o nó master do MRS via SSH
2. Executar a CLI do Spark SQL:

```
spark-sql --master yarn \  
  --conf spark.sql.hive.convertMetastoreOrc=true \  
  -e "SELECT COUNT(*) FROM poc_db.customers;"
```

3. Verificar se a contagem retornada corresponde ao esperado (10.000)
4. Executar uma consulta de amostra para verificar a integridade dos dados:

```
spark-sql --master yarn \  
  -e "SELECT customer_id, name FROM poc_db.customers LIMIT 5;"
```

5. Confirmar que a consulta retorna 5 linhas com valores não nulos

Resultado Esperado

1. COUNT(*) retorna 10.000.
2. A consulta de amostra retorna 5 linhas com valores válidos de customer_id e name.

Resultado do Teste

Atende

Observações

Se a CLI do Spark SQL não estiver disponível, utilizar a interface web do componente Spark2x do MRS para submeter a consulta.

2.3 Segurança e Controle de Acesso

Caso de Teste 7: Verificar aplicação de política IAM nos buckets OBS

Objetivo

Confirmar que as políticas IAM restringem corretamente o acesso ao bucket OBS — usuários autorizados podem ler/escrever enquanto usuários não autorizados são negados, e a política do bucket segue o princípio do menor privilégio.

Escopo do Teste

Aplicação de política IAM; Controle de acesso OBS; Validação de menor privilégio

Pré-requisitos

1. Bucket OBS poc-datalake-raw existe com a política IAM designada.
2. Dois usuários IAM de teste: um com função **OBS_ReadWrite**, outro sem permissões OBS.
3. CLI obsutil configurada para ambos os usuários.

Procedimento

1. Usando o usuário autorizado, carregar um arquivo de teste em poc-datalake-raw
2. Usando o usuário autorizado, baixar e verificar o conteúdo do arquivo
3. Usando o usuário não autorizado, tentar carregar um arquivo — verificar se a requisição é negada (HTTP 403)
4. Usando o usuário não autorizado, tentar listar objetos no bucket — verificar se a requisição é negada
5. Revisar a política do bucket no Console em **Console** → **Object Storage Service** → **Bucket Policies** e confirmar que concede apenas as permissões pretendidas

```
# Usuário autorizado – carregar e verificar
obsutil cp test-file.txt obs://poc-datalake-raw/test-file.txt
obsutil ls obs://poc-datalake-raw/ -limit=5

# Usuário não autorizado – esperar erros 403
obsutil cp test-file.txt obs://poc-datalake-raw/unauthorized.txt
obsutil ls obs://poc-datalake-raw/
```

Resultado Esperado

1. Usuário autorizado consegue carregar e baixar com sucesso.
2. Usuário não autorizado recebe HTTP 403 nas operações de carregamento e listagem.
3. Política do bucket concede apenas as permissões de leitura/escrita pretendidas às funções designadas.

Resultado do Teste

Não atende

Observações

Verificar os logs do Cloud Trail se a negação de acesso não for retornada conforme esperado — pode haver herança de política do projeto ou domínio.

Nota: Alterações em políticas IAM podem levar até 5 minutos para propagar. Aguardar antes de retestar após modificar uma política.

Caso de Teste 8: Verificar regra de mascaramento de dados do DataArts Studio**Objetivo**

Confirmar que colunas de dados sensíveis (ex.: campos PII) são mascaradas conforme a regra de mascaramento do DataArts Studio quando consultadas por usuários não administradores.

Escopo do Teste

Regras de mascaramento de dados; Proteção de PII; Acesso a dados de não-admin

Pré-requisitos

1. Caso de Teste 4 concluído (workspace do DataArts Studio acessível).
2. Regra de mascaramento configurada para a coluna `customers.ssn` (Número de Seguro Social).
3. Usuário de teste possui função **Dayu_User** (não administrador).

Procedimento

1. No console do DataArts Studio, navegar até **Arquitetura de Dados** → **Mascaramento de Dados**
2. Verificar se a regra de mascaramento para `customers.ssn` está ativa e utiliza a estratégia **Mascarar Tudo**
3. Consultar a tabela `customers` como usuário de teste via console SQL do DataArts Studio
4. Verificar se a coluna `ssn` retorna valores mascarados (ex.: `*****`) em vez dos dados reais
5. Consultar a mesma tabela como usuário administrador e verificar se os valores reais são retornados

Resultado Esperado

1. Regra de mascaramento ativa para `customers.ssn`.
2. Usuário não administrador visualiza valores mascarados na coluna `ssn`.
3. Usuário administrador visualiza valores reais na coluna `ssn`.

Resultado do Teste

Atende com ressalvas

Observações

Resultado com ressalvas: as regras de mascaramento são aplicadas no momento da consulta — os dados subjacentes no OBS/HDFS não são modificados, de modo que o mascaramento não cobre exportações diretas do bucket.

2.4 Configuração de Ambiente e Ferramentas

Caso de Teste 9: Verificar implantação de infraestrutura via Terraform

Objetivo

Confirmar que a configuração Terraform implanta todos os recursos de infraestrutura necessários (VPC, MRS, OBS, DataArts Studio) sem erros e que o arquivo de estado reflete os recursos esperados.

Escopo do Teste

Infraestrutura como Código; Implantação via Terraform; Verificação de estado de recursos

Pré-requisitos

1. CLI terraform instalada (versão ≥ 1.0).
2. Credenciais AK/SK configuradas via variáveis de ambiente ou *provider.tf*.
3. Arquivos de configuração Terraform no diretório do projeto.

Procedimento

1. Navegar até o diretório do projeto Terraform
2. Executar `terraform init` para inicializar o diretório de trabalho
3. Executar `terraform plan` e revisar as alterações planejadas — confirmar que todos os recursos esperados estão listados
4. Executar `terraform apply -auto-approve` e aguardar a conclusão
5. Executar `terraform output` e verificar se todos os valores de saída estão preenchidos
6. Executar `terraform state list` e confirmar que a contagem de recursos corresponde à arquitetura

```
# Inicializar e implantar infraestrutura
cd /opt/poc-infra/terraform
terraform init
terraform plan -out=tfplan
terraform apply tfplan

# Verificar implantação
terraform output
terraform state list | wc -l
```

Resultado Esperado

1. `terraform apply` conclui sem erros.

2. Todos os valores de saída estão preenchidos (ID do cluster, nome do bucket, ID da VPC).
3. Contagem de recursos no estado corresponde à especificação da arquitetura.

Resultado do Teste

Atende

Observações

Se `terraform apply` falhar, verificar as credenciais AK/SK e os limites de cota da região `sa-brazil-1`.

Importante

Não pule a etapa de revisão do `terraform plan`. Aplicar alterações não revisadas em ambientes adjacentes à produção pode causar perda de dados ou interrupção de serviços.

Caso de Teste 10: Verificar carregamento de dados de teste no OBS

Objetivo

Confirmar que todos os arquivos de dados de teste necessários (CSV, JSON) podem ser carregados no bucket OBS designado e são acessíveis pelo pipeline do DataArts Studio.

Escopo do Teste

Carregamento de dados no OBS; Preparação de dados de teste; Conectividade do pipeline de dados

Pré-requisitos

1. Caso de Teste 2 concluído (bucket OBS acessível).
2. Arquivos de dados de teste preparados no diretório local `./test-data/`.
3. CLI `obsutil` configurada com as credenciais do usuário de teste.

Procedimento

1. Listar os arquivos de dados de teste locais e verificar sua integridade (contagem de linhas, esquema)
2. Carregar todos os arquivos no bucket OBS raw usando `obsutil`
3. Verificar se cada arquivo existe no OBS listando o conteúdo do bucket
4. Baixar um arquivo de amostra do OBS e compará-lo com o original local (checksum)
5. No DataArts Studio, verificar se a conexão de fonte de dados consegue ler os arquivos carregados

```
# Carregar arquivos de dados de teste no OBS
obsutil cp ./test-data/customers.csv
obs://poc-datalake-raw/customers.csv
obsutil cp ./test-data/transactions.json
obs://poc-datalake-raw/transactions.json

# Verificar carregamentos
obsutil ls obs://poc-datalake-raw/ -limit=20
```

Resultado Esperado

1. Todos os arquivos de dados de teste são carregados sem erros.
2. A listagem do bucket mostra todos os arquivos carregados com tamanhos corretos.
3. O checksum do arquivo baixado corresponde ao original local.
4. A conexão de fonte de dados do DataArts Studio consegue ler os arquivos.

Resultado do Teste

Não testado

Observações

Arquivos grandes (>1 GB) devem ser carregados usando o recurso de upload multipart do OBS para maior confiabilidade.

Nota: Consulte a [Documentação do Huawei Cloud OBS](#) para opções avançadas de upload e solução de problemas.

[Espaço reservado para imagem]

Arquivo: assets/test-execution-dashboard.png

Descrição: Captura de tela do painel de execução de testes mostrando os resultados de todos os casos de teste

Coloque o arquivo de imagem no caminho acima e substitua este comando por

`\image{assets/test-execution-dashboard.png} ou`
`\imagecap{assets/test-execution-dashboard.png}{caption}.`

3

Conclusão

ID	Título	Status
1	Verificar implantação e saúde do cluster MRS	Atende
2	Verificar criação e acessibilidade do bucket OBS	Atende
3	Verificar configuração de VPC e grupo de segurança	Atende
4	Verificar criação do workspace do DataArts Studio	Atende
5	Verificar execução do pipeline de ingestão de dados	Atende
6	Verificar consulta Spark SQL no cluster MRS	Atende
7	Verificar aplicação de política IAM nos buckets OBS	Não atende
8	Verificar regra de mascaramento de dados do DataArts Studio	Atende com ressalvas
9	Verificar implantação de infraestrutura via Terraform	Atende
10	Verificar carregamento de dados de teste no OBS	Não testado

3.1 Diagramas de Arquitetura de Teste

Informação

Esta seção e a seguinte (“Paleta de Cores da Marca”) são conteúdo de demonstração do sample — não fazem parte de um documento de teste real.

Diagramas podem ser usados para visualizar fluxos e arquiteturas de teste.

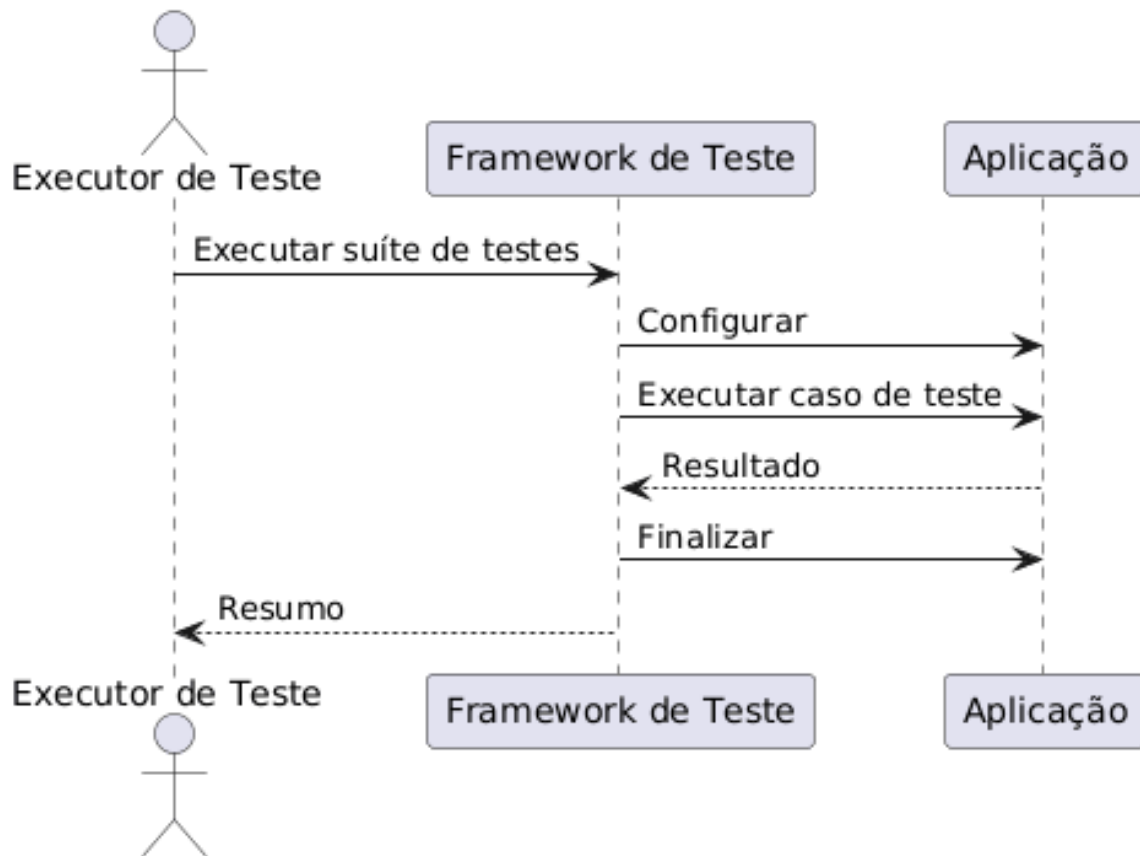


Diagrama 1: Diagrama de sequência da execução de teste

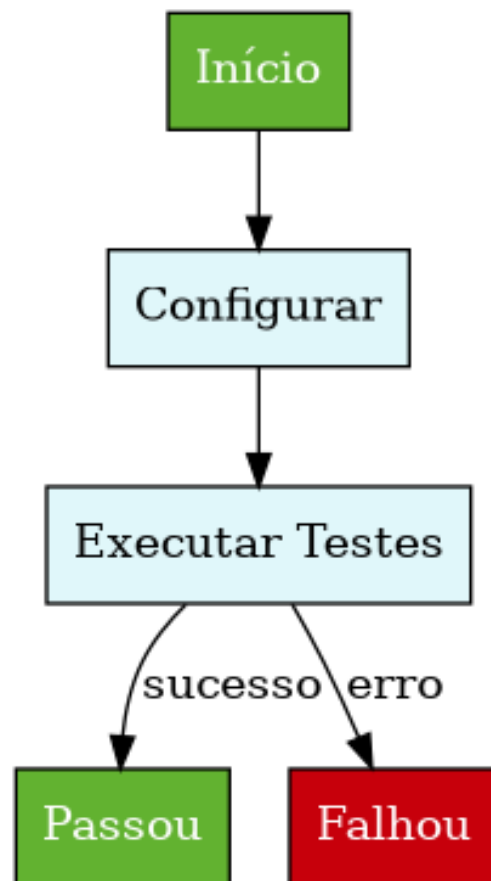


Diagrama 2: Fluxograma de teste

3.2 Paleta de Cores da Marca

O template define cores auxiliares da marca alinhadas às Diretrizes de Marca da Huawei Cloud:

Cor	HEX	Pantone	Amostra
Rose Red	#C40054	7636	
Dark Red	#7F0001	483C	
Orange	#ED6D00	165C	
Yellow	#FCC800	7406C	
Green	#62B230	3501C	
Blue	#30B5C5	2227C	

4

Histórico de versões

2.4.0 2026-09-28

- Adicionada coluna “Amostra” na tabela de critérios de aceitação, mostrando os badges coloridos (\testresultbadge) que correspondem aos usados nos casos de teste e na tabela de resumo.

2.3.1 2026-09-28

- Adicionada nota indicando que as seções de diagramas e paleta de cores são conteúdo de demonstração do sample.

2.3.0 2026-09-28

- Removidos os pré-requisitos duplicados do bloco de objetivos — agora apenas o objetivo geral aparece no bloco; os pré-requisitos permanecem na seção dedicada “Pré-condições e Preparações”.
- Corrigida a capitalização do texto da capa: “CO., LTD” → “Co., Ltd.”

2.2.0 2026-09-23

- Vocabulário de resultados unificado: a tabela de critérios de aceitação agora usa “Atende”, “Atende com ressalvas”, “Não atende” e “Não testado”.
- Resultado “Bloqueado” substituído por “Atende com ressalvas” (Partial): o Caso de Teste 8 e a linha correspondente da tabela de resumo foram convertidos, com observações alinhadas ao resultado parcial.

2.1.2 2026-09-22

- Revisão de idioma: cabeçalhos de tabela “Cor”/“Amostra” (antes “Color”/“Sample”).

2.1.1 2026-09-19

- Corrigidas as datas do histórico de versões — \today substituído pelas datas reais de lançamento.

2.1.0 2026-09-13

- Reestruturação do documento para layout de 3 seções: Introdução, Casos de Teste (com subseções por domínio) e Conclusão (com tabela de resumo dos testes).
- Remoção dos prefixos manuais TC-XXX dos títulos dos casos de teste — o ambiente os numera automaticamente como “Caso de Teste 1:”, “Caso de Teste 2:”, etc.
- Atualização dos IDs do testsummary de TC-XXX para numeração automática 1–10.
- Alteração de teststeps de tabela tabular para parágrafos auto-numerados. \teststep agora aceita 1 argumento (ação apenas). Imagens, blocos de código e callouts podem ser inseridos livremente entre os passos.

2.0.0 2026-09-13

- Layout de casos de teste redesenhado: barras de cabeçalho mini de largura total para cada campo, ambiente testprocedure com tabela de passos de 2 colunas, ordem dos campos atualizada (Resultado do Teste antes de Observações).

1.1.0*2026-09-12*

- Adicionadas demonstrações abrangentes de recursos: caixas de destaque `warning`, `tip`, `infobox`; blocos `code`; `badge`; `weblink`; `note`; `param`; `image`, `imagecap` e `imageplaceholder`; caminhos `menu`; e referências `inlinecode`.
- Adicionados casos de teste de Segurança e Controle de Acesso.
- Adicionados casos de teste de Configuração do Ambiente e Ferramentas.

1.0.0*2026-09-12*

- Versão inicial.
 - Adicionados casos de teste de Arquitetura da Plataforma.
 - Adicionados casos de teste de Engenharia de Dados.
-